

→javascript

→html is a markup language

→css is a styling language

→javascript is a programming language

Using the console:

Uses repl

Repl:read-evaluate-print-loop

Read→reads input

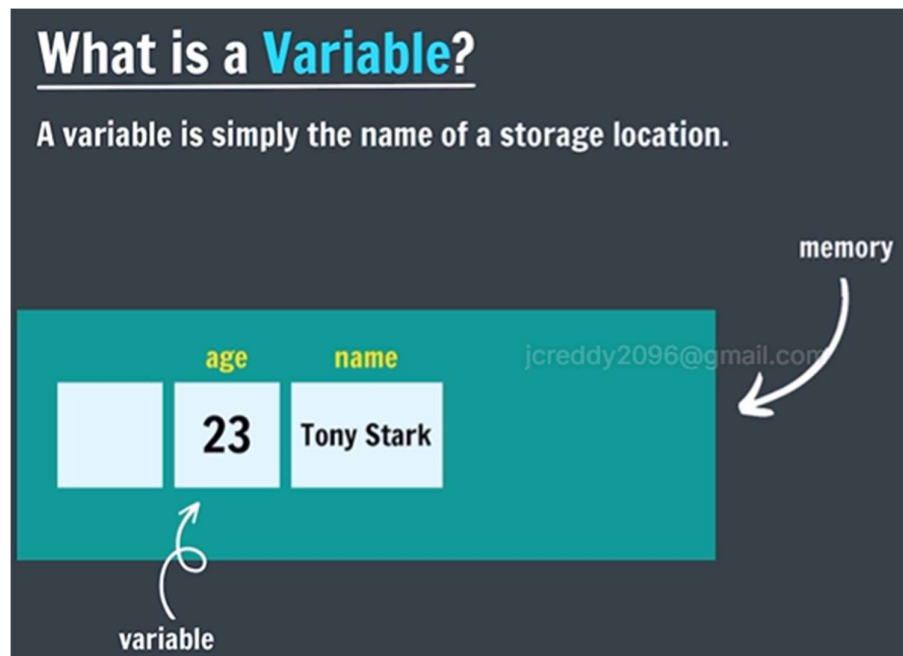
Evaluate→calculation

Print→it prints

Loop→repeat

→by clicking ctrl+L it will clear the warnings in the console

→what is a variable?



→variable means we change the value of the variable

Ex: a=10,b=20

a=30;

name= "tony stark"

→ data types in js

Data Types in JS

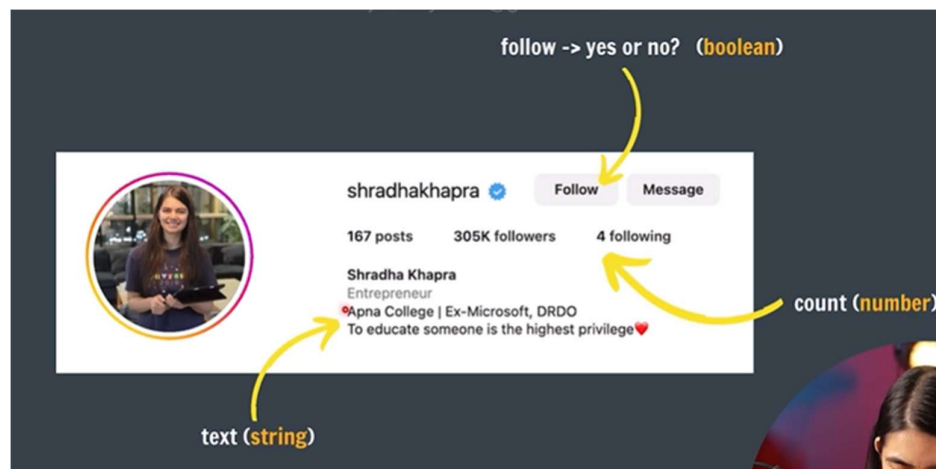
Primitive Types

- Number
- Boolean
- String
- Undefined
- Null
- BigInt
- Symbol

In console if we type `typeof(a)` `a` is a variable by this we can know the data type of the

```
3 Issues: 3  
> typeof 5.9  
< 'number'  
> typeof 'a'  
< 'string'  
>
```

variable.



→ numbers in js

- Positive (14) & Negative (-4)
- Integers (45, -50)
- Floating numbers - with decimal (4.6, -8.9)

We can do the operations in js

```
> a=10
```

10

> b=5

5

$$> a+b$$

◀ 15

 $\gt a-b$

5

$$> a*b$$

50

$$> a/b$$

2

> a%b

 $\angle \theta$ $a+b, a-b, a/b, a*b$

> 0.999999999999999

◀ 0.999999999999999

> 0.999999999999999999999999999999999999

< 1

→ operations in js

Operations in JS

```
a = 20
b = 10

//addition
sum = a + b

//subtraction
diff = a - b

//multiplication
prod = a * b

//division
div = a / b

//modulo
rem = a % b
```

- Modulo (remainder operator)
 $12 \% 5 = 2$
- Exponentiation (power operator)
 $2^{**}3 = 8$

Handwritten diagram: $a + b$ with arrows pointing to 'a' and 'b' labeled 'operands' and an arrow pointing to '+' labeled 'operator'.

APNA COLLEGE

JS

jcreddy2096@gmail.com



$a+b$

a,b are operands

+ represents the operation

→ to get the remainder of any number we find the modulo(%) of that number

→ by using the modulo operator we use to find whether the number is even or odd number

→ for an even number we get final =0;

→ for an odd number we get =1;

Ex: $4\%2=0$, $3\%2=1$

→ Exponentiation(power operator)

$a^{**}b$

$a=2, b=3$

$2^{**}3=8$

→ NaN in js

NaN is Not A Number

The NaN global property is a value representing **Not-A-Number**.

$0 / 0$

$\text{NaN} - 1$

$\text{NaN} * 1$

$\text{NaN} + \text{NaN}$

```
> 0 / 0
< NaN
> typeof NaN
< 'number'
> NaN + 1
< NaN
> NaN - 1
< NaN
> NaN * 1
< NaN
> NaN * NaN
< NaN
>
```

→ Operator precedence

→ in operator precedence we take the precedence based on the level top to bottom and then if they are at same level like $*, /, \%$ at that time we need to move left to right to solve.

Left to right

Operator Precedence

This is the general **order** of solving an expression.

$()$
↑
 $**$
↑
 $*, /, \%$
↑
 $+, -$

BODMAS

$(5+2) / 7 + 1 * 2$
 $(7) / 7 + 1 * 2$
 $1 + 2$
 $\rightarrow$
 $= 3$

$$(2 + 1) * 3$$

$$3 / 1 + 2 ** 2$$

$$4 + 1 * 6 / 2$$

→let keyword

let keyword

Syntax of declaring variables

```
let age = 23 ;  
age = age + 1 ;  
age = 23 + 1 = 24  
  
let num1 = 1 ;  
let num2 = 2 ;  
let sum = num1 + num2 ;
```

```
let cgpa ;  
cgpa = 8.9  
jcreddy
```

```
> let num1=1;  
< undefined  
> let num2=2;  
< undefined  
> let sum=num1+num2;  
< undefined  
> sum  
< 3
```

→const keyword

const keyword

values of **constants** can't be changed with re-assignment & they can't be re-declared

```
const year = 2025;  
year = 2026 // Error  
year = year + 1 // Error  
  
const pi = 3.14 ;  
const g = 9.8 ;  
jcreddy2096@gmail.com
```

→once we declare the variable under const it value cannot be changed through the program .

→once its value is fixed that's it.

→in let we can change the values at any line of the program by using the let for any variable.

```
> const pi = 3.14;
< undefined
> let rad = 4;
< undefined
> let area = pi * r ** 2;
✖ ▶ Uncaught ReferenceError:
    at <anonymous>:1:17
> let area = pi * rad ** 2;
< undefined
> area
< 50.24
```

→var keyword: var keyword works as it is like the let keyword but the var is outdated usage.

var keyword

Old Syntax of writing variables

```
var age = 23 ;
```

```
var cgpa = 8.9 ;
```

```
var num1 = 1 ;
```

```
var num2 = 2 ;
```

```
var sum = num1 + num2 ;
```


→ practice question

Practice Qs

Qs. What is the value of age after this code runs?

```
let age = 23 ;  
age + 2; //after 2 years
```

23 ✓

assignment operators

Qs. What is the value of avg after this code runs?

```
let hindi = 80;  
let eng = 90;  
let math = 100;  
let avg = (hindi + eng + math) / 3;
```

> let age=23;
< undefined
> age+2;
< 25
> age
< 23

→ assignment operators

Assignment Operators

age = age + 1

age += 1

age = age - 1

age -= 1

age = age * 1

age *= 1

age = age / 2
age /= 2 ;

age = age % 4;
age %= 4;

→ unary operators

Unary Operators

Handwritten note: binary operators 2 operands

<code>age = age + 1</code>	<code>age = age - 1</code>	$a + b$
<code>age += 1</code>	<code>age -= 1</code>	$a \times b$
<code>age++</code> // increment	<code>age--</code> // decrement	a / b
		$a = b$

Unary Operators

Pre-increment (Change, then use)

```
let age = 10 ;  
let newAge = ++age ;
```

Post-increment (Use, then change)

```
let age = 10 ;  
let newAge = age++ ;
```

Handwritten notes:
`++a` (pre-inc) `--a` (pre-dec)
`a++` (post-inc) `a--` (post-dec)

```
> let a = 5;  
< undefined  
> ++a;  
< 6  
> a  
< 6  
> a++;  
< 6  
> a  
< 7
```

→ practice qs

Practice Qs

Qs. What is the value of each variable in each line of code?

```
let num = 5;  
let newNum = num++;  
newNum = ++num;
```

Handwritten annotations:

- ← `num = 5` ✓
- ← `newNum = 5` `num = 6`
- ← `newNum = 7` `num = 7`

Handwritten email: jcreddy2096@gmail.com

→ identifier rules

Identifier Rules

All JavaScript variables must be identified with **unique names** (identifiers).

- Names can contain ^{A-Z} letters, ^{a-z} digits, ⁰⁻⁹ underscores, and ^{\$} dollar signs. (no space)
- Names must begin with a letter.
- Names can also begin with \$ and _.
- Names are case sensitive (y and Y are different variables).
- Reserved words (like JavaScript keywords) CANNOT be used as names.

camelCase

Way of writing identifiers

camelCase (JS naming convention)

camelCase

let totalPrice

let fullName;

snake_case

let full_name;

PascalCase

let FullName;

→ boolean in js

Boolean in JS

Boolean represents a truth value -> **true or false** / yes or no

```
let age = 23 ;
```

```
let isAdult = true ;
```

```
let age = 13 ;
```

```
let isAdult = false ;
```

```

> let a = 5;
< undefined
> typeof a
< 'number'
> a = true;
< true
> typeof a
< 'boolean'
>

```

```

> let age = 13;
< undefined
> let isAdult = false;
< undefined
> age = 25;
< 25
> isAdult = true;
< true
> isAdult
< true
> isAdult = false;
< false
> isAdult
< false
> typeof isAdult
< 'boolean'
>

```

```

> isAdult = True;
✖ ▶ Uncaught ReferenceError:
  at <anonymous>:
> isAdult = 'True';
< 'True'
> typeof isAdult
< 'string'
> true
< true
> false
< false

```

→ in js we can also change the type of a variable by changing its value in js unlike c++, java from program starting to end same type variable we use but in js we can change.

What is TypeScript?

Static Typed, where JS is dynamic typed

fixed *change*

Designed by Microsoft



TS

a = 5 ✓

a = true X ← JS

a = 10 ✓

→ practice questions

Qs. Find the errors in the following code

a) `let ①age = 5;` ←
`let 2age = 10;` ←

b) `let marks = 75;`
`let isPass = True;`

c) `let isPass = 'true';`

Qs. Find the errors in the following code ?

- a) `let 1age = 5;` ← `let age1 = 5;`
`let 2age = 10;` ← `let age2 = 10;`
- b) `let marks = 75;` ←
`let isPass = True;` ← true
- c) `let isPass = 0true0;` { ' ' } string
↑ boolean true

→ String in js

String in JS

Strings are **text** or sequence of characters

→ word, para

`let name = "Tony Stark";`

`let role = 'ironman';`

`let char = 'a';`

`let num = '23';`

`let empty = " ";`

```
> let sentence = 'this is "apple"';  
< undefined  
> sentence  
< 'this is "apple"'
```

→ in strings we can use the single quotes (') and double quotes (") for writing the strings.

→ `let a = ""` → it is also a string

Let `b = " "` → it is also a string

```
top Filter
> let sentence = 'this is "apple"';
< undefined
> sentence
< 'this is "apple"'
> let sentence = "this is 'apple'";
< undefined
> sentence
< "this is 'apple'"
> let sentence = 'this is 'apple'';
✖ Uncaught SyntaxError: Unexpected identifier 'apple'
> let sentence = "this is "apple"";
✖ Uncaught SyntaxError: Unexpected identifier 'apple'
>
```

→string indices

→index=position

String Indices

let name = "TONY STARK";

T	O	N	Y		S	T	A	R	K
0	1	2	3	4	5	6	7	8	9

name[0] -> 'T'
name[1] -> 'O' ...

```
top Filter
> let name = "TONY STARK";
< undefined
> name
< 'TONY STARK'
> name[0]
< 'T'
> name[1]
< 'O'
> name[2]
< 'N'
> name[3]
< 'Y'
> name[4]
< ' '
> "shradha".length
< 7
> "shradha"[6]
< 'a'
> "shradha"[0]
< 's'
>
```


→String length

→to know the length of the variable name.length

→string concatenation

Concatenation

adding strings together

"tony" + " " + "stark" = "tony stark"

"tony" + 1 = "tony1"

```
> "tony" + " " + "stark"
< 'tony stark'
> let firstName = "tony";
< undefined
> let lastName = "stark";
< undefined
> let fullName = firstName + lastName;
< undefined
> fullName
< 'tonystark'
> fullName = firstName + " " + lastName;
< 'tony stark'
> fullName
< 'tony stark'
> "tony" + 1
< 'tony1'
```

→null & undefined in js

null & undefined in JS

undefined

A variable that has not been assigned a value is of type undefined.

```
> let a;
< undefined
> a
< undefined
```

null

The null value represents the **intentional** absence of any object value.

To be explicitly assigned.

```
> let a = null;
< undefined
> a
< null
```

```
> let name;
< undefined
> name
< undefined
> typeof name
< 'undefined'
> name = "apnacollege";
< 'apnacollege'
> name[1000];
< undefined
> let a = null;
< undefined
> a
< null
>

> let year = null;
< undefined
> year
< null
> year = 2000;
< 2000
> year
< 2000
```

→undefined : means js doesn't know the value of that variable

→null: this value we assigned intentionally in program we assign another value to it

→null is a keyword.

→practice questions

Practice Qs

Qs. Declare your name as a string and print its length in JS.

Qs. Declare your first name as a string and print its first character.

Qs. Declare your first name as a string and print its last character.

Qs. What is output of following code :

```
"apnacollege"+123|
```

Qs. What are lengths of an empty string & a string with a single space?


```
let name="subbu"
```

```
undefined
```

```
name
```

```
'subbu'
```

```
name.length
```

```
5
```

```
let firstname="jayachandra"
```

```
undefined
```

```
firstname
```

```
'jayachandra'
```

```
firstname[firstname.length-1]
```

```
'a'
```

```
firstname[0];
```

```
'j'
```

```
let n=""
```

```
> let "apnacollege"+123;
```

```
undefined
```

```
✖ Uncaught SyntaxError: Unexpected string
```

```
n.length
```

```
> let "apnacollege"+123
```

```
0
```

```
✖ Uncaught SyntaxError: Unexpected string
```

```
let a=" ";
```

```
> let n="apnacollege"+123
```

```
undefined
```

```
↳ undefined
```

```
a.length
```

```
> n
```

```
1
```

```
↳ 'apnacollege123'
```

```
>
```

→js-2

```
Console.log("hello");
```

console.log()

To write (log) a message on the console

```
console.log("Apna College");
```

```
console.log(1234);
```

```
console.log(2+2);
```

```
console.log("Apna", "College", 123);
```

```
Hello!  
< undefined  
> console.log("Hello World from Apna College!");  
Hello World from Apna College!  
< undefined  
> let num = 123;  
< undefined  
> console.log("num");  
num  
< undefined  
> console.log(num);  
123  
< undefined  
> console.log(1+2);  
3  
< undefined  
> console.log("Hello", "World", num, (1+5));  
Hello World 123 6  
< undefined  
>
```

→js file doesn't directly connected to browser by using the console.log we use to print the values in browser using the console.log()

→linking js file



The correct position to add the script tag in the html file is not in the head part it is in the body tag at the ending of the html code we add the script tag.